

Introduction to prep

prep is an R package that supports application of a machine-learning-based extension of the condensed mastery profiles standard setting method (Clark et al. 2017). prep provides functionality for assigning attribute mastery profiles to panelists, fitting a logistic regression model to pinpoint the range around inflection points between adjacent performance levels following the rangefinding round, fitting a machine learning model based on panelist ratings to classify each profile into one of the performance levels, predicting performance levels of profiles using the machine learning model, evaluation the performance of the machine learning model, and conducting a validation exercise. Here, you'll find a brief overview of how to use the prep package to apply the machine-learning-based standard setting method.

```
library(prep)
```

Identifying Possible Profiles

We begin with identifying which attribute mastery profiles that are possible. In practice, this will be determined based on the assessment design. For example, adaptive routing may render some attribute mastery profiles impossible if the mastery of the highest level of an attribute may route students to a higher level on the next attribute and thus some mastery patterns involving lower levels are not possible). For the purposes of this vignette, we created the set of possible profiles based on all possible combinations of attribute mastery. In this example, we have an assessment where eight attributes are measured, and each attribute is measured at four levels, resulting there are five possible mastery scores for each attribute: No mastery, Mastery of Level 1, Mastery of Level 2, Mastery of Level 3, Mastery of Level 4.

```
possible_profiles <- readRDS(here::here("vignettes/possible_profiles_rds"))

possible_profiles
#> # A tibble: 390,625 x 8
#>   att.1 att.2 att.3 att.4 att.5 att.6 att.7 att.8
#>   <int> <int> <int> <int> <int> <int> <int> <int>
#> 1     0     0     0     0     0     0     0     0
#> 2     0     0     0     0     0     0     0     1
#> 3     0     0     0     0     0     0     0     2
#> 4     0     0     0     0     0     0     0     3
#> 5     0     0     0     0     0     0     0     4
#> 6     0     0     0     0     0     0     1     0
#> 7     0     0     0     0     0     0     1     1
#> 8     0     0     0     0     0     0     1     2
#> 9     0     0     0     0     0     0     1     3
#> 10    0     0     0     0     0     0     1     4
#> # 390,615 more rows
```

Observed Profiles

Next, the profiles observed during the administration of the assessment are loaded. The observed profiles object should include a column for each of the assessed attributes (i.e., the attribute mastery statuses = att.1, att.2, and so on), a column for the number of times each profile was observed (e.g., n), and a column for the proportion of students with the observed profile (e.g., prop).

```
observed <- readRDS(here::here("vignettes/observed_rds"))

observed
#> # A tibble: 351,538 x 10
#>   att.1 att.2 att.3 att.4 att.5 att.6 att.7 att.8 n prop
#>   <int> <int> <int> <int> <int> <int> <int> <int> <dbl> <dbl>
#> 1     1     0     0     0     0     0     0     0 0.5647 0.00000292
#> 2     0     0     0     0     0     0     0     0 1.0555 0.00000457
#> 3     0     0     0     0     0     0     0     0 3.9305 0.00000481
#> 4     0     0     0     0     0     0     0     0 4.8310 0.00000429
#> 5     0     0     0     0     0     0     1     0 0.2426 0.00000125
#> 6     0     0     0     0     0     0     1     1 1.6346 0.00000216
#> 7     0     0     0     0     0     0     1     2 2.5115 0.00000161
#> 8     0     0     0     0     0     0     1     3 3.2409 0.00000124
#> 9     0     0     0     0     0     0     1     4 1.991 0.00000103
#> 10    0     0     0     0     0     0     2     0 0.5275 0.00000272
#> # 351,528 more rows
```

Assigning Profiles for the Rangefinding Round

Given the observed profiles, profiles are assigned to panelists using assign_profiles(). This function takes user-specified information about the panel design as the input, and it outputs the panelists' profile assignments.

For this example, we are using a table-based panel design. Specifically, there are five tables (i.e., num_assignment_groups = 5) with four panelists per table (i.e., panelists_per_table = 4). The function balances profile assignment so that one profile is seen by all panelists at all of the tables (i.e., shared_profiles = 1) and the panelists at each table have two-thirds of the assigned profiles in common (i.e., proportion_of_shared_profiles = .67). To demonstrate, with three profiles assigned to each panelist at each level of the total levels mastered, one profile will be seen by all of the panelists, and two of the three assigned profiles assigned to each panelist, which includes the profile seen by all panelists, will be common to all panelists at the table. If the user wants to assign the same set of profiles to all panelists at a table, this can be done with proportion_of_shared_profiles = 1. The user also specifies the values of the total number of skills mastered to be used for assignment (i.e., included_totals = c(5L, 10L, 15L, 20L, 25L, 30L)) and the number of profiles to assign per level (i.e., profiles_per_level = 3). The total number of skills mastered is based on the total number of levels mastered across the attributes measured by the assessment. For instance, with eight assessed attributes with four levels each (e.g., scores of 0, 1, 2, 3, or 4 are possible for each attribute), the possible levels mastered ranges from 0 (no mastery of any level for any attribute) to 32 (i.e., mastery of all levels for all attributes). included_totals specifies the profiles to retain for assignment based on the sum of levels mastered across all attributes. In this example, we specified that profiles with 5, 10, 15, 20, 25, and 30 levels mastered should be retained and that each panelist should be assigned three profiles at each of these levels. The observed_count_label parameter specifies the field name for the observed profiles' sample sizes (e.g., "n"), and the observed_proportion_label parameter specifies the field name for the observed profiles' proportions (e.g., "prop").

```
final_assignments <- assign_profiles(
  num_assignment_groups = 5L,
  table_configuration = list(
    panelists_per_table = 4L,
    proportion_of_shared_profiles = .67
  ),
  observed = observed,
  observed_count_label = "n",
  observed_proportion_label = "prop",
  shared_profiles = 1L,
  included_totals = c(5L, 10L, 15L, 20L, 25L, 30L),
  profiles_per_level = 3L,
  output_dir = here::here("vignettes")
)

final_assignments
#> # A tibble: 180 x 14
#>   att.1 att.2 att.3 att.4 att.5 att.6 att.7 att.8 total rater1 rater2
#>   <int> <int> <int> <int> <int> <int> <int> <int> <dbl> <dbl> <dbl>
#> 1     1     0     0     0     0     0     0     0 5 table1 0 1
#> 2     1     3     0     0     1     0     0     0 5 table1 0 1
#> 3     1     2     0     0     1     0     1     0 5 table3 1 0
#> 4     1     1     0     0     3     0     0     0 5 table2 0 0
#> 5     1     0     1     0     0     3     0     0 5 table5 0 1
#> 6     1     0     0     0     3     0     1     0 5 table3 0 0
#> 7     1     0     0     0     0     3     0     1 5 table1 0 0
#> 8     1     0     0     0     0     3     0     1 5 table4 1 1
#> 9     0     0     4     1     0     0     0     0 5 table2 0 0
#> 10    0     3     1     0     0     0     1     0 5 table5 1 1
#> # 179 more rows
#> # 12 more variables: rater3 <int>, rater4 <int>
```

The output of assign_profiles() includes a column for each measured attribute, the total number of skills mastered, table number (when applicable), and a column for each rater. In the column for each rater, values of 1 indicate the panelist is assigned the profile, and values of 0 indicate the panelist is not assigned the profile. In a table-based design, there is one column for each rater at each table. In other words, rater1 at table1 is different than rater1 at table2.

Instead of a table-based design, we could assign profiles directly to panelists. The following example assigns profiles to 10 panelists, where one of profile is common to all panelists (i.e., shared_profiles = 1). The panelist-based design uses the same settings for included_totals and profiles_per_level. Note that the table_configuration is not provided to assign_profiles(), indicating a panelist-based design. Consequently, there is no table field in the output when using a panelist-based design.

```
final_assignments <- assign_profiles(
  num_assignment_groups = 10L,
  observed = observed,
  observed_count_label = "n",
  observed_proportion_label = "prop",
  shared_profiles = 1L,
  included_totals = c(5L, 10L, 15L, 20L, 25L, 30L),
  profiles_per_level = 3L,
  output_dir = here::here("vignettes")
)

final_assignments
#> # A tibble: 360 x 12
#>   profile_num att.1 att.2 att.3 att.4 att.5 att.6 att.7 att.8 rater table_id
#>   <int> <int> <int> <int> <int> <int> <int> <int> <dbl> <dbl> <dbl>
#> 1     1     1     3     0     0     0     0     0 0 table1 rater1
#> 2     1     1     3     0     0     2     0     0 0 table3 rater2
#> 3     1     1     3     0     0     2     0     0 0 table3 rater3
#> 4     1     1     3     0     0     2     0     0 0 table3 rater4
#> 5     2     2     1     3     0     0     0     0 1 table1 rater1
#> 6     2     2     1     3     0     0     0     0 1 table1 rater2
#> 7     2     2     1     3     0     0     0     1 0 table1 rater3
#> 8     2     2     1     3     0     0     0     2 0 table1 rater4
#> 9     3     3     1     3     0     0     0     0 1 table2 rater1
#> 10    3     3     1     3     0     0     0     1 0 table2 rater2
#> # 359 more rows
#> # 1 more variable: rating <dbl>
```

Identifying Profile Ranges for Pinpointing

fit_lnr() fits a logistic regression model to identify a range around the inflection points between adjacent performance levels based on the ratings from the rangefinding round. This function receives the panelists' ratings, the performance level labels, the number of levels per attribute, and the output directory as input. The function automatically saves the ranges of potential inflection points to the output directory as pp-range.csv; however, it also returns the ranges, which we save in pinpoint_ranges. pinpoint_ranges contains a column specifying the cut-point, along with columns for the predicted inflection point, the lower bound for the range around the inflection point, and the upper bound for the range around the inflection point. For example, the range around the predicted inflection point for the cut-point between the Emerging and Approaching the Target performance levels labelled as "Approaching the Target" in pinpoint_ranges is 6 skills mastered with a lower bound of 3 skills mastered and an upper bound of 12 skills mastered.

```
pl_labels <- c("Emerging", "Approaching the Target", "At Target", "Advanced")

pinpoint_ranges <- fit_lnr(
  ratings,
  pl_labels,
  att_levels = 4,
  output_dir = here::here("vignettes")
)

pinpoint_ranges
#> # A tibble: 3 x 4
#>   cut_point predicted pinpoint_min pinpoint_max
#>   <chr> <dbl> <dbl> <dbl>
#> 1 Advanced 23 20 27
#> 2 Approaching the Target 8 3 12
#> 3 At Target 17 14 21
```

Assigning Profiles for the Pinpointing Round

The range around the inflection points are used to retain observed profiles for possible assignment in the pinpointing round. Then, the previously assigned profiles should be removed from the set of possible profiles. We retain the possible profiles where the total number of levels mastered is within the identified range around each inflection point with profiles_to_retain(). The profiles eligible for pinpointing assignment (i.e., the output of profiles_to_retain()) and the observed profiles are used to assigned to panelists using assign_profiles(). Of note, the profiles assigned in the rangefinding round (i.e., assigned_profiles = ratings) are input into assign_profiles() when assigning profiles for the pinpointing round to ensure these profiles are not assigned again. In the example, the design for the pinpointing round is largely similar to the design of the rangefinding round. The primary difference is that only two profiles are assigned at each level during pinpointing (i.e., profiles_per_level = 2) whereas three profiles were assigned at each level during rangefinding. The format of the output remains the same.

```
retain_totals <- profiles_to_retain(
  pinpoint_ranges,
  num_attributes = 8,
  attribute_levels = 4
)

final_assignments <- assign_profiles(
  num_assignment_groups = 5L,
  table_configuration = list(
    panelists_per_table = 4L,
    proportion_of_shared_profiles = .67
  ),
  observed = observed,
  observed_count_label = "n",
  observed_proportion_label = "prop",
  shared_profiles = 1L,
  included_totals = retain_totals,
  profiles_per_level = 2L,
  assigned_profiles = ratings,
  output_dir = here::here("vignettes")
)

final_assignments
#> # A tibble: 600 x 14
#>   att.1 att.2 att.3 att.4 att.5 att.6 att.7 att.8 total rater1 rater2
#>   <int> <int> <int> <int> <int> <int> <int> <int> <dbl> <dbl> <dbl>
#> 1     1     3     0     0     0     0     0     0 3 table5 0 0
#> 2     1     1     3     0     0     1     0     0 3 table3 0 0
#> 3     1     1     0     0     1     0     0     0 3 table2 0 0
#> 4     1     0     1     0     0     1     0     0 3 table2 1 0
#> 5     1     1     0     0     2     0     0     0 3 table3 1 0
#> 6     1     0     1     1     0     0     0     1 3 table2 0 1
#> 7     1     0     0     0     0     0     2     0 3 table1 1 1
#> 8     1     0     0     0     0     0     2     0 3 table2 1 1
#> 9     1     0     0     0     0     0     2     0 3 table3 1 1
#> 10    1     0     0     0     0     0     2     0 3 table4 1 1
#> # 599 more rows
#> # 12 more variables: rater3 <int>, rater4 <int>
```

As with profile assignment for the rangefinding round, we could also use a panelist-based design for the pinpointing round. Like the rangefinding round, this example assigns profiles to 10 panelists and the remaining settings are consistent with the table-based design.

```
final_assignments <- assign_profiles(
  num_assignment_groups = 10L,
  observed = observed,
  observed_count_label = "n",
  observed_proportion_label = "prop",
  shared_profiles = 1L,
  included_totals = retain_totals,
  profiles_per_level = 2L,
  assigned_profiles = ratings,
  output_dir = here::here("vignettes")
)

final_assignments
#> # A tibble: 1,320 x 11
#>   att.1 att.2 att.3 att.4 att.5 att.6 att.7 att.8 rater rating
#>   <int> <int> <int> <int> <int> <int> <int> <int> <dbl> <dbl>
#> 1     1     3     0     0     2     0     0     0 0 table3 rater1 2
#> 2     1     3     0     0     2     0     0     0 0 table3 rater2 1
#> 3     1     3     0     0     2     0     0     0 0 table3 rater3 1
#> 4     1     3     0     0     2     0     0     0 0 table3 rater4 1
#> 5     1     1     3     0     0     0     1     0 0 table1 rater1 1
#> 6     1     1     3     0     0     0     1     0 0 table1 rater2 1
#> 7     1     1     3     0     0     0     1     0 0 table1 rater3 1
#> 8     1     1     3     0     0     0     1     0 0 table1 rater4 2
#> 9     1     1     3     0     0     0     1     0 0 table2 rater1 1
#> 10    1     1     3     0     0     0     1     0 0 table2 rater2 2
#> # 1,319 more rows
```

Fitting the Machine Learning Models

We fit the machine learning model to the panelist rating data using fit_ml(). Users define the machine learning workflow, using models with functionality supported by tidymodels (Kuhn & Wickham, 2020). When the model involves hyperparameters, the workflow can specify that the hyperparameters may be tuned or the hyperparameters can be pre-specified. The workflow, panelist rating data, observed profiles, and possible profiles are included as input to fit_ml(). Users also specify the number of levels per attribute (i.e., att_levels), the number of performance levels (i.e., num_pls), the label for the panelist ratings column in the panelist rating data (i.e., rating_id = "rating"), the label for the counts in the observed profiles (i.e., observed_count_label = "n"), the label for the proportions in the observed profiles (i.e., observed_proportion_label = "prop"), the desired agreement metrics (i.e., metrics = c("accuracy"), and the output directory. Output from fit_ml() is an S7 class that includes the fitted model, the in-sample agreement metrics, and the out-of-sample agreement metrics.

```
workflow <-
  parsnip::rand_forest(
    ntry = 1,
    trees = parsnip::tune(),
    mtry = parsnip::tune()
  ) >|>
  parsnip::set_mode("classification") >|>
  parsnip::set_engine("ranger")

workflow <-
  parsnip::rand_forest(ntry = 1, trees = 100, mtry_n = 1) >|>
  parsnip::set_mode("classification") >|>
  parsnip::set_engine("ranger")

possible_profiles <- readRDS(here::here("vignettes/possible_profiles_rds"))

fitted_model <- fit_ml(
  user_workflow = workflow,
  ratings_data,
  observed,
  possible_profiles,
  att_levels = 4,
  num_pls = 4,
  rating_id = "rating",
  observed_count_label = "n",
  observed_proportion_label = "prop",
  metrics = c("accuracy", "auc", "kappa", "osagreement"),
  output_dir = here::here("vignettes")
)
```

Model Evaluation

The fitted_model object includes information to evaluate model performance. The model agreement output includes a data frame with the requested evaluation metrics for in-sample and out-of-sample agreement, with one row per metric. The in-sample agreement can be accessed using fitted_model\$in_sample_agreement, and the out-of-sample agreement can be accessed using fitted_model\$out_of_sample_agreement.

The supported evaluation metrics are accuracy (i.e., classification accuracy), adjacent (i.e., adjacent classification accuracy), kappa (i.e., Cohen's kappa), auc (i.e., area under the receiver operating characteristic curve [ROC AUC]), and the proportion of students with the assigned profiles. Other than assignment, the evaluation metrics are intended to quantify how well the fitted model recovered the panelists' ratings. The assignment metrics are only included in fit_in_sample_agreement as these are only intended to reflect whether the profiles used to train the model generally reflect the distribution of observed profiles. For this example, focusing first on the in-sample agreement (i.e., fitted_model\$in_sample_agreement), the classification accuracy (i.e., the percentage of panelist ratings that were accurately predicted) was approximately 77%, with a ROC AUC of .93 and a Cohen's kappa of .70. These performance metrics indicate acceptable agreement between the model predictions and panelist ratings, indicating a successful application of the method. However, the in-sample agreement is based on how well the model fits the data used to fit the model, and these metrics may not be the best reflection of how well the model generalizes. The assignment metrics in fit_in_sample_agreement allow for a comprehensive evaluation of model generalizability as it pertains to model training. For instance, these statistics allow the user to evaluate whether a substantial percentage of students were represented in the assigned profiles. To demonstrate, in this example, we randomly assigned observed profiles with a uniform distribution, meaning profiles were generally observed with the same level of prevalence. Consequently, the assigned profiles represent a small proportion of students, which may have implications for generalizability if we were to observe this proportion of students represented in the assigned profiles in an actual standard setting. Users will have to define their own evaluation thresholds to determine whether the machine learning models demonstrate sufficient evidence of generalizability, accuracy, and reliability as well as whether the models are trained on profiles reflecting an adequate proportion of the student population.

```
fitted_model$in_sample_agreement
#> # A tibble: 6 x 3
#>   metric estimator estimate
#>   <chr> <chr> <dbl>
#> 1 accuracy multiclass 0.766
#> 2 roc_auc hand_till 0.924
#> 3 cohens_kappa multiclass 0.70
#> 4 profiles_assigned <NA> 608
#> 5 students_with_assigned_profile <NA> 3983482.
#> 6 props_students_within_assigned_profile <NA> 0.00206
```

Focusing now on the out-of-sample agreement (i.e., fitted_model\$out_of_sample_agreement), the classification accuracy (i.e., the percentage of panelist ratings that were accurately predicted) was approximately 73%, with a ROC AUC of .89 and a Cohen's kappa of .63. Of note, it is expected that the evaluation metrics will reflect poorer performance for the out-of-sample agreement as these reflect performance with profiles that the model has not seen before. Consequently, out-of-sample agreement is a better reflection of model generalizability. Estimates approaching the in-sample agreement rates indicate increasing model generalizability. For example, out-of-sample classification accuracy of 73% compared to in-sample classification accuracy of 77% would indicate better generalizability than if the out-of-sample classification accuracy had been 65%.

```
fitted_model$out_of_sample_agreement
#> # A tibble: 3 x 3
#>   metric estimator estimate
#>   <chr> <chr> <dbl>
#> 1 accuracy multiclass 0.726
#> 2 roc_auc hand_till 0.877
#> 3 cohens_kappa multiclass 0.626
```

Model Validation

A validation step is necessary to gather evidence that the model is producing reasonable performance level predictions since the model can likely only be trained on panelist rating data reflecting a small subset of the observed profiles. In the validation exercise, previously unseen profiles are shown to panelists along with the model-predicted performance levels. The panelists are then asked whether they agree with the model-predicted performance level.

First, a new set of unseen profiles must be assigned to the panelists. This set of profiles is assigned similar to assignment for the range finding and pinpointing rounds, with the exception that all previous assignments are input into assign_profiles() so that only new profiles are assigned during the validation exercise. Notably, the standard setting design (e.g., number of panelists) should not change from range finding and pinpointing to the validation exercise, but there may be design choices pertaining to which profiles to assign (e.g., the number of profiles to assign at each level).

```
validation_assignments <- assign_profiles(
  num_assignment_groups = 3L,
  table_configuration = list(
    panelists_per_table = 4L,
    proportion_of_shared_profiles = .67
  ),
  observed = observed,
  observed_count_label = "n",
  observed_proportion_label = "prop",
  shared_profiles = 1L,
  included_totals = c(5L, 10L, 15L, 20L, 25L),
  profiles_per_level = 2L,
  assigned_profiles = ratings,
  output_dir = here::here("vignettes")
)

validation_assignments
#> # A tibble: 125 x 14
#>   att.1 att.2 att.3 att.4 att.5 att.6 att.7 att.8 total rater1 rater2
#>   <int> <int> <int> <int> <int> <int> <int> <int> <dbl> <dbl> <dbl>
#> 1     1     3     0     0     1     0     0     0 5 table5 0 1
#> 2     2     2     0     1     1     0     0     0 5 table5 0 1
#> 3     2     0     1     1     0     0     0     1 5 table3 0 1
#> 4     2     0     0     2     0     0     0     1 5 table5 0 0
#> 5     1     3     0     0     1     0     0     0 5 table2 0 0
#> 6     1     1     0     0     1     0     2     0 5 table2 0 1
#> 7     1     0     1     2     0     0     1     0 5 table3 0 0
#> 8     1     0     0     4     0     0     0     0 5 table2 0 0
#> 9     1     0     0     1     1     0     1     1 5 table3 1 0
#> 10    0     2     0     0     0     0     3     0 5 table2 1 0
#> # 115 more rows
#> # 12 more variables: rater3 <int>, rater4 <int>
```

Once the set of profiles for the validation exercise have been identified, performance levels can be assigned to each of the profiles in the validation exercise using assign_pl(). Prior to inputting validation_assignments into assign_pl(), we need to insert a dummy column for rating (i.e., mutate(rating = NA)). Further, we need to remove total from validation_assignments (i.e., select(-"total")), pivot the data into long-form (i.e., tidyr::longer(cols = dplyr::starts_with("rater")), names_to = "rater_id", values_to = "assigned")), filter down to only the assigned profiles (i.e., filter(exclude = 1)), and remove assigned (i.e., select(-"assigned")). Because this is the validation exercise, it is only reflecting in the model ratings, so we limit the output of assign_pl() using \$model_ratings.

```
validation_pl <- assign_pl(
  fitted_model = fitted_model,
  validation_assignments = validation_assignments,
  dplyr::mutate(rating = NA) >|>
  dplyr::select(-"total") >|>
  tidyr::pivot_longer(cols = dplyr::starts_with("rater")),
  names_to = "rater_id",
  values_to = "assigned") >|>
  dplyr::filter(assigned == 1) >|>
  dplyr::select(-"assigned")

possible_profiles,
att_levels = 4,
num_pls = 4,
rating_id = "rating",
observed_count_label = "n",
observed_proportion_label = "prop",
output_dir = here::here("vignettes")
)

validation_ratings <-
  readRDS(here::here("vignettes/validation_ratings_rds"))

validation_agreement <- eval_agreement(
  model_ratings = validation_ratings,
  metrics = c("accuracy", "auc", "kappa"),
  observed,
  num_pls = 4,
  rating_id = "rating",
  observed_count_label = "n",
  observed_proportion_label = "prop",
  output_dir = here::here("vignettes")
)

validation_agreement
#> # A tibble: 3 x 3
#>   metric estimator estimate
#>   <chr> <chr> <dbl>
#> 1 accuracy multiclass 0.870
#> 2 roc_auc hand_till 0.931
#> 3 cohens_kappa multiclass 0.821
```

References

Clark, A. K., Nash, E., Karvonen, M., & Kingston, N. (2017). Condensed mastery profile method for setting standards for diagnostic assessment systems. *Educational Measurement: Issues and Practice*, 36(4), 5–15. <https://doi.org/10.1111/emip.12182>

Kuhn, M., & Wickham, H. (2020). *Tidymodels: A collection of packages for modeling and machine learning using tidyverse principles*. <https://www.tidymodels.org>